
GI41 高度ゲームアルゴリズム

第8回 講義テーマ

ランレングス符号による圧縮

講義内容

1. ゲームにおける圧縮の必要性
 2. 圧縮のコンセプト
 3. 圧縮の方向性
 4. ゲームソフトにおける圧縮
 5. 連長圧縮(ランレングス圧縮)
 6. ランレングス展開の高速化
-

1. ゲームにおける圧縮の必要性

1. ゲームにおける圧縮の必要性

- メモリの制限

- コンシューマー機は総じてメモリ容量が少ない(コストダウンのため)。

- メディアからのローディング

- データはメディアからメモリにロードしなくては利用できない

- ＞少しでもローディング時間を短くするために圧縮を利用する

2. 圧縮のコンセプト

2. 圧縮のコンセプト

- サイズ重視

とにかく圧縮されたファイルのサイズをできるだけコンパクトにすることが重要視される

- 速度重視

展開にリアルタイム性が必要な場合、デコード（復元）処理ができるだけ高速に実行できることが重要視される

3. 圧縮の方向性

3. 圧縮の方向性

- 可逆圧縮 (lossless compression)
データの欠落がまったく起こらない圧縮方式。

可逆圧縮方式により圧縮された符号を復号すると、圧縮前のデータを完全に復元できる。

3. 圧縮の方向性

プログラムや文字データなどはデータが1ビットでも欠けるとまったく異なった結果になることから、圧縮を行なう場合には必ず可逆圧縮を用いる必要がある。

このため、ファイル全般を取り扱う汎用の圧縮プログラムはすべて可逆圧縮となっている。

3. 圧縮の方向性

- 非可逆圧縮 (lossy compression)

多少のデータの欠落を許容する代わりに、劇的に圧縮効率を高めた圧縮方式。

非可逆圧縮方式により圧縮されたデータを復号しても、圧縮前のデータを完全に復元することはできない。

3. 圧縮の方向性

非可逆圧縮は画像や映像、音声などの圧縮に用いられる。これらのデータはプログラムや文字データと違って、多少のデータの欠損や改変があっても、まったく意味が変わってしまうということはない。このため、人間の視聴覚特性を元に人間が認識しにくい成分を削ったり、見えない/聞こえないよう巧妙にデータを改変したりして、極めて効率のよい圧縮を実現している。

4. ゲームソフトにおける圧縮

4. ゲームソフトにおける圧縮

ゲームソフトにおける圧縮では用途に応じて複数のフォーマットを使い分けているのが現状である。

4. ゲームソフトにおける圧縮

(プログラム、データの圧縮)

- 当然「可逆圧縮」でなければならない。どちらかというと圧縮サイズよりも展開速度重視。

(映像、音声データの圧縮)

- 「非可逆圧縮」が主に用いられる。標準規格を採用してコストダウンを図る傾向が見られる。
-

4. ゲームソフトにおける圧縮

4. ゲームソフトにおける圧縮

ゲームソフトにおける圧縮では用途に応じて複数のフォーマットを使い分けているのが現状である。

5. 連長圧縮(ランレングス圧縮)

5. 連長圧縮(ランレングス圧縮)

(考え方)ある連続したデータを、「そのデータ1つ分と連続した長さで表現する」ことで圧縮

(例)「A A A A A B B B B B B B B B A A A」

これは「A 5 B 9 A 3」と表せる。＜ Aが5回続き、そのあとにBが9回、そしてAが3回続いている

5. 連長圧縮(ランレングス圧縮)

(欠点と解決方法)

欠点 … データが連続していないと符号化後のデータが元のデータより膨らんでしまう

(例)「A B C D A B C A B A B C D E」

これは「A 1 B 1 C 1 D 1 A 1 B 1 C 1 A 1 B 1 A 1 B 1 C 1 D 1 E 1」となってしまいます。

(この場合の圧縮率は200%、つまり元の2倍)

5. 連長圧縮(ランレングス圧縮)

解決方法の一例 … ある一定数だけ同じ
データが繰り返した場合にだけランレングス圧縮を施す

(例) 元のデータ「A B C D D D D」

通常の方法	対策済みの符号化
-------	----------

A 1 B 1 C 1 D 4	A B C D D 2
-----------------	-------------

6. ランレングス展開の高速化

6. ランレングス展開の高速化

通常ランレングス圧縮されているデータをメモリ上に展開する場合の処理は、同じデータがある領域に連続コピーすることに他ならないので、一般的にはfor文なのでループを構成してポインタをずらしながらコピーしていく形をとることになる。

6. ランレングス展開の高速化

```
for(count = 0; count < COPY_COUNT; count++) {  
    *dstpt++ = data;  
}
```

6. ランレングス展開の高速化

ループ回数にリミットがあるとしてその回数が数百回程度であるなら、ループ展開を行ってしまうという大胆な方法を採用することも可能である。この方法を使うと、展開速度が劇的に向上する。

6. ランレングス展開の高速化

```
switch(count){  
  case 255:  *dstpt++ = data;  
  case 254:  *dstpt++ = data;  
  case 253:  *dstpt++ = data;  
  case 252:  *dstpt++ = data;  
  
    :      :  
  
  case 1:    *dstpt++ = data;  
}
```
